

**Q1.** The intervals and corresponding frequencies are as follows. age frequency

1-5 200

5-15 450

15-20 300

20-50 1500

50-80 700

80-110 44

Compute an approximate median value for the data.

**R CODE:**

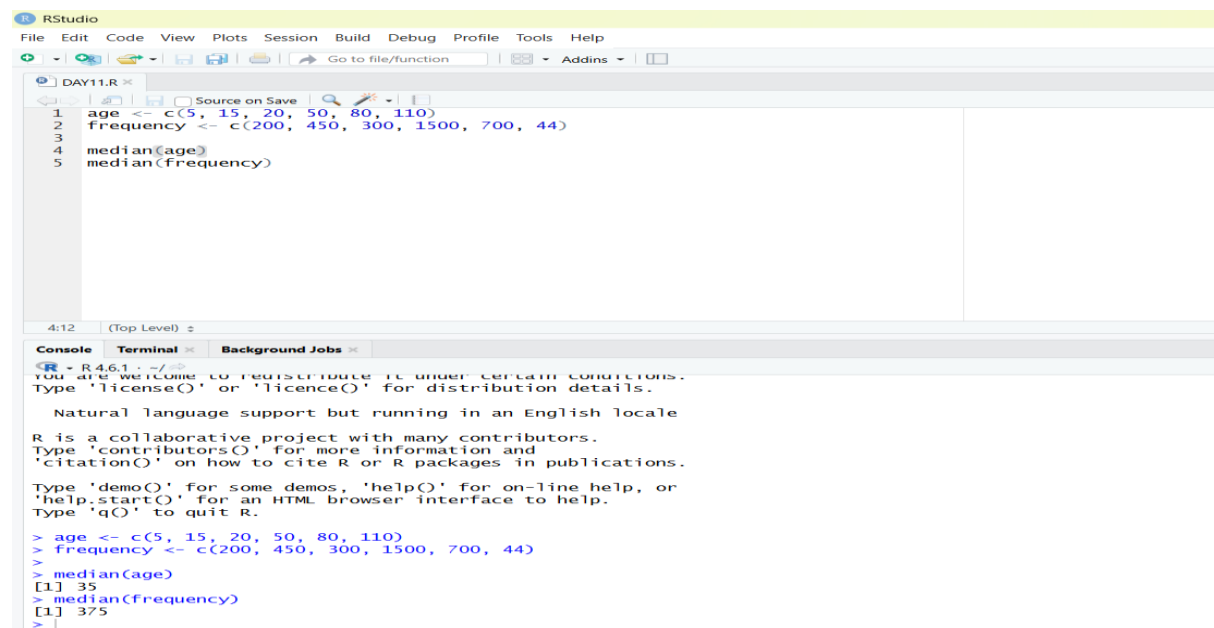
```
age <- c(5, 15, 20, 50, 80, 110)
```

```
frequency <- c(200, 450, 300, 1500, 700, 44)
```

```
median(age)
```

```
median(frequency)
```

**OUTPUT:**



The screenshot shows the RStudio interface. The script editor contains the following code:

```
1 age <- c(5, 15, 20, 50, 80, 110)
2 frequency <- c(200, 450, 300, 1500, 700, 44)
3
4 median(age)
5 median(frequency)
```

The console shows the output of the code:

```
> age <- c(5, 15, 20, 50, 80, 110)
> frequency <- c(200, 450, 300, 1500, 700, 44)
> median(age)
[1] 35
> median(frequency)
[1] 375
>
```

**Q2.** Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order) 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70.

(a) What is the mean of the data? What is the median?

(b) What is the mode of the data? Comment on the data's modality (i.e., bimodal, trimodal, etc.).

(c) What is the midrange of the data?

**(d) Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data?**

**R CODE:**

```
age <- c(13,15,16,16,19,20,20,21,22,22,25,25,25,25,
        30,33,33,35,35,35,35,36,40,45,46,52,70)

mean(age)

median(age)

mode_age <- names(table(age))[table(age) == max(table(age))]

mode_age

range(age)

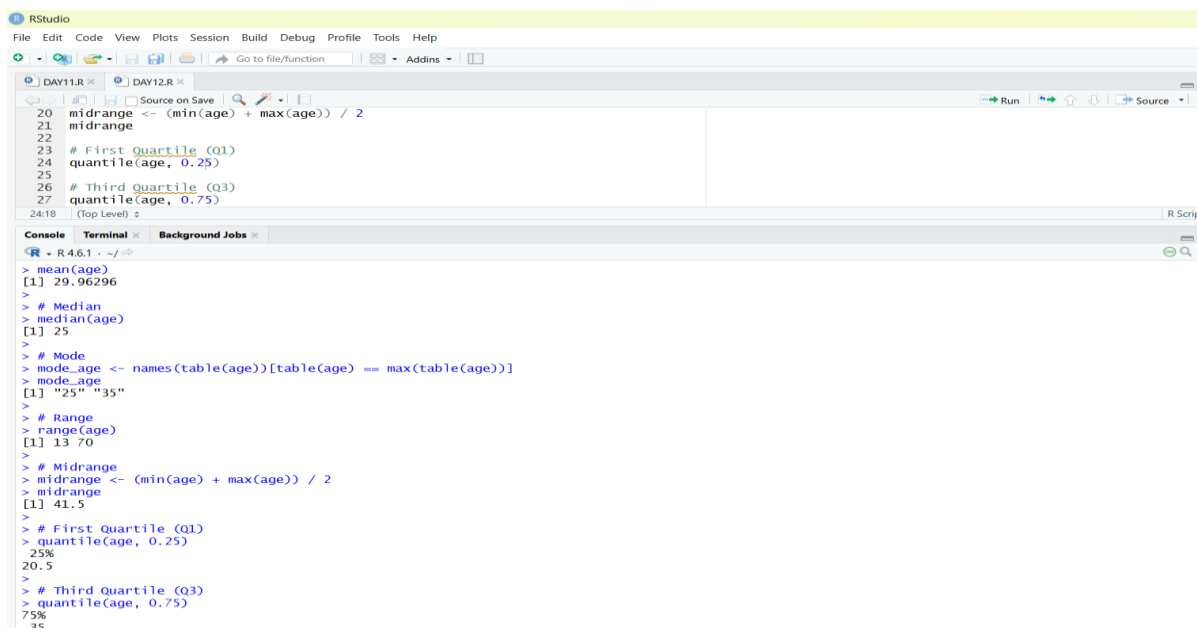
midrange <- (min(age) + max(age)) / 2

midrange

quantile(age, 0.25)

quantile(age, 0.75)
```

**OUTPUT:**



The screenshot shows the RStudio interface. The script editor contains the following code:

```
20 midrange <- (min(age) + max(age)) / 2
21 midrange
22
23 # First Quartile (Q1)
24 quantile(age, 0.25)
25
26 # Third Quartile (Q3)
27 quantile(age, 0.75)
```

The console shows the output of the code:

```
> mean(age)
[1] 29.96296
> # Median
> median(age)
[1] 25
> # Mode
> mode_age <- names(table(age))[table(age) == max(table(age))]
> mode_age
[1] "25" "35"
> # Range
> range(age)
[1] 13 70
> # Midrange
> midrange <- (min(age) + max(age)) / 2
> midrange
[1] 41.5
> # First Quartile (Q1)
> quantile(age, 0.25)
25%
20.5
> # Third Quartile (Q3)
> quantile(age, 0.75)
75%
35
```

### **Q3. Normalization (Reduction & Transformation)**

Use the two methods below to normalize the following group of data: 200, 300, 400, 600, 1000

**(a) min-max normalization by setting min = 0 and max = 1**

**(b) z-score normalization**

**R CODE:**

```
x <- c(200, 300, 400, 600, 1000)
```

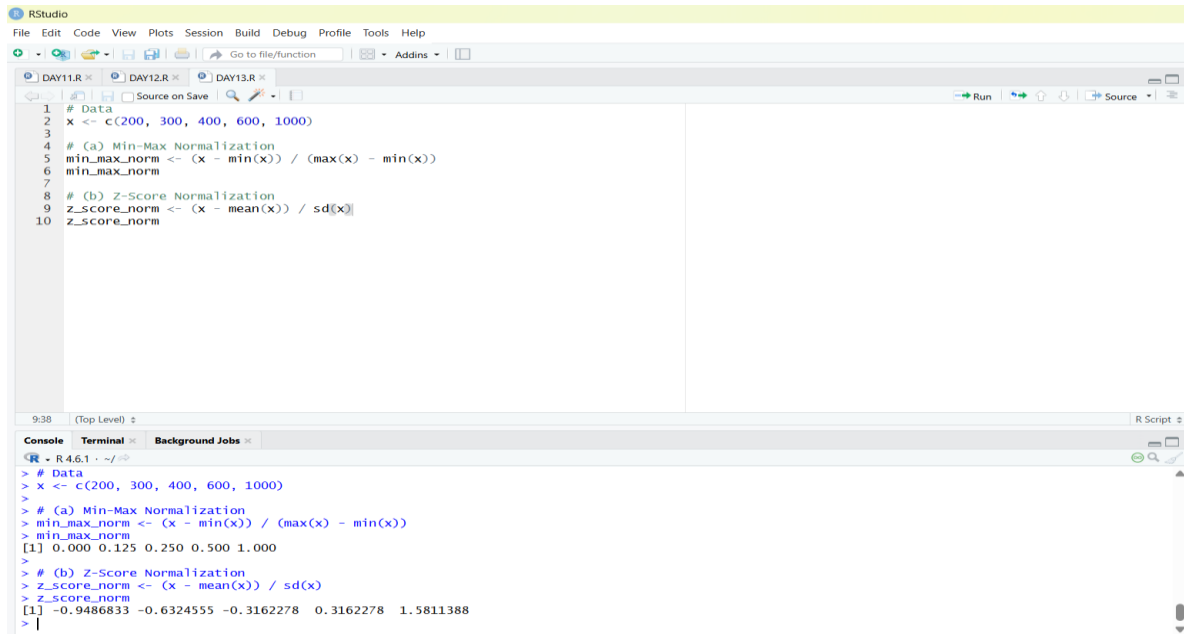
```
min_max_norm <- (x - min(x)) / (max(x) - min(x))
```

```
min_max_norm
```

```
z_score_norm <- (x - mean(x)) / sd(x)
```

```
z_score_norm
```

## OUTPUT:



```
# Data
x <- c(200, 300, 400, 600, 1000)

# (a) Min-Max Normalization
min_max_norm <- (x - min(x)) / (max(x) - min(x))
min_max_norm

# (b) Z-Score Normalization
z_score_norm <- (x - mean(x)) / sd(x)
z_score_norm
```

```
R 4.6.1 ~ /
> # Data
> x <- c(200, 300, 400, 600, 1000)
>
> # (a) Min-Max Normalization
> min_max_norm <- (x - min(x)) / (max(x) - min(x))
> min_max_norm
[1] 0.000 0.125 0.250 0.500 1.000
>
> # (b) Z-Score Normalization
> z_score_norm <- (x - mean(x)) / sd(x)
> z_score_norm
[1] -0.9486833 -0.6324555 -0.3162278  0.3162278  1.5811388
> |
```

## Q4. Smoothing by Binning(Reduction & Transformation)

Data: 11,13,13,15,15,16,19,20,20,20,21,21,22,23,24,30,40,45,45,45,71,72,73,75

a) Smoothing by bin mean

b) Smoothing by bin median

c) Smoothing by bin boundaries

### R CODE:

```
data <- c(11,13,13,15,15,16,19,20,20,20,21,21,
```

```
22,23,24,30,40,45,45,45,71,72,73,75)
```

```
bins <- 5
```

```
bin_indices <- cut(data, bins)
```

```
mean_smooth <- tapply(data, bin_indices, mean)
```

```
print(mean_smooth)
```

```
median_smooth <- tapply(data, bin_indices, median)
```

```
print(median_smooth)
```

```
boundary_smooth <- tapply(data, bin_indices, function(x) {
```

```

c(min(x), max(x))
})
print(boundary_smooth)

```

## OUTPUT:

```

19
20 # (c) Smoothing by Bin Boundaries
21 boundary_smooth <- tapply(data, bin_indices, function(x) {
22   c(min(x), max(x))
23 })
24 print(boundary_smooth)
23:3 (Top Level)

```

```

> # (a) Smoothing by Bin Mean
> mean_smooth <- tapply(data, bin_indices, mean)
> print(mean_smooth)
[10.9,23.8] [23.8,36.6] [36.6,49.4] [49.4,62.2] [62.2,75.1]
17.78571 27.00000 43.75000 NA 72.75000
> # (b) Smoothing by Bin Median
> median_smooth <- tapply(data, bin_indices, median)
> print(median_smooth)
[10.9,23.8] [23.8,36.6] [36.6,49.4] [49.4,62.2] [62.2,75.1]
19.5 27.0 45.0 NA 72.5
> # (c) Smoothing by Bin Boundaries
> boundary_smooth <- tapply(data, bin_indices, function(x) {
+   c(min(x), max(x))
+ })
> print(boundary_smooth)
$`[10.9,23.8]`
[1] 11 23

$`[23.8,36.6]`
[1] 24 30

$`[36.6,49.4]`
[1] 40 45

$`[49.4,62.2]`
NULL

$`[62.2,75.1]`
[1] 71 75

```

**Q5.** Suppose that a hospital tested the age and body fat percentage (%fat) data for 18 randomly selected adults with the following values:

|      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|
| age  | 23   | 23   | 27   | 27   | 39   | 41   | 47   | 49   | 50   |
| %fat | 9.5  | 26.5 | 7.8  | 17.8 | 31.4 | 25.9 | 27.4 | 27.2 | 31.2 |
| age  | 52   | 54   | 54   | 56   | 57   | 58   | 58   | 60   | 61   |
| %fat | 34.6 | 42.5 | 28.8 | 33.4 | 30.2 | 34.1 | 32.9 | 41.2 | 35.7 |

- Calculate the mean, median, and standard deviation of age and %fat.
- Draw the boxplots for age and %fat.
- Draw a scatter plot and a Q-Q plot based on these two variables.

## R CODE:

```

age <- c(23,23,27,27,39,41,47,49,50,52,54,54,56,57,58,58,60,61)
fat <- c(9.5,26.5,7.8,17.8,31.4,25.9,27.4,27.2,31.2,
        34.6,42.5,28.8,33.4,30.2,34.1,32.9,41.2,35.7)

mean(age)
median(age)

```

```

sd(age)

mean(fat)

median(fat)

sd(fat)

boxplot(age, fat,
        names = c("Age", "Body Fat"),
        main = "Boxplots of Age and Body Fat",
        ylab = "Values")

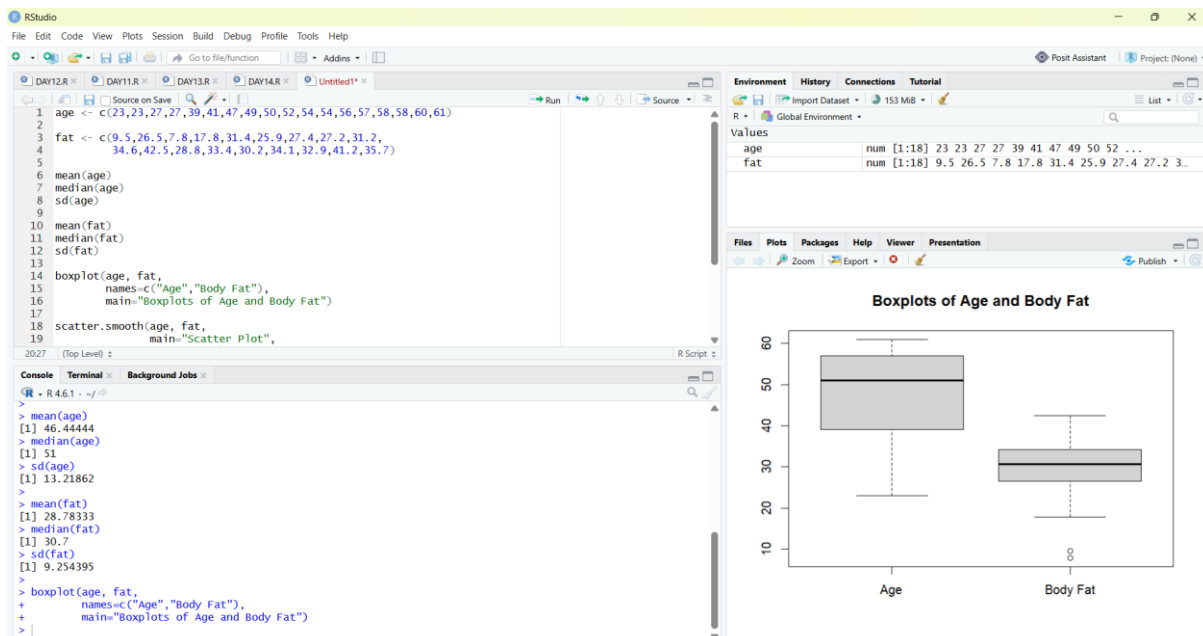
scatter.smooth(age, fat,
               main = "Scatter Plot of Age vs Body Fat",
               xlab = "Age",
               ylab = "Body Fat (%)")

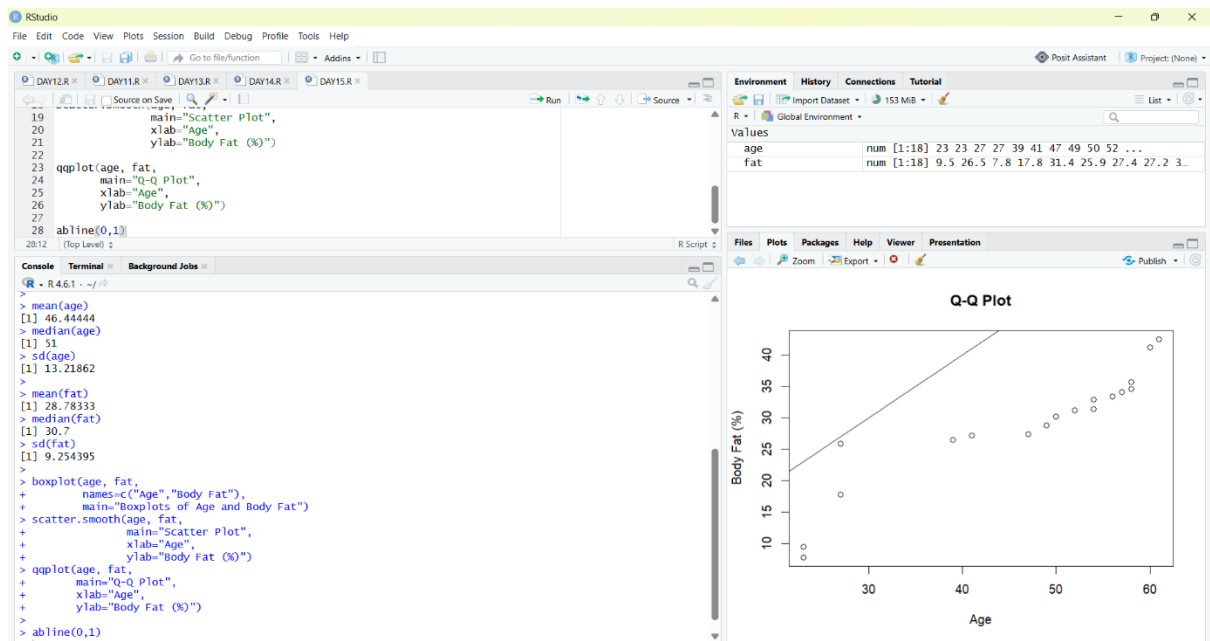
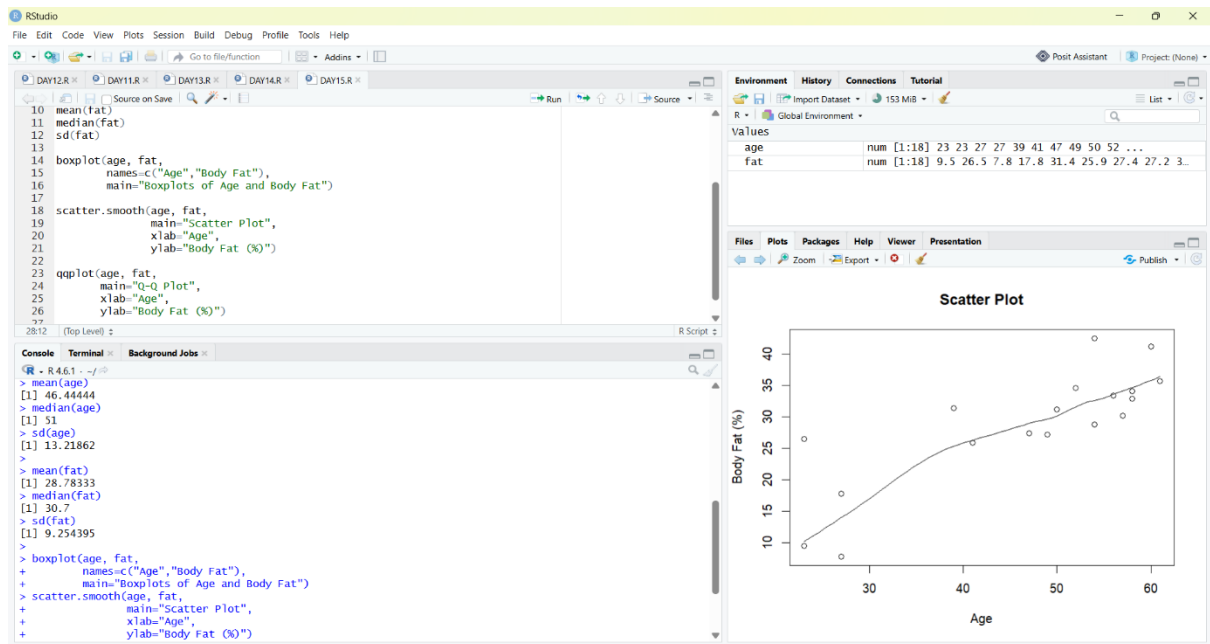
qqplot(age, fat,
       main = "Q-Q Plot of Age and Body Fat",
       xlab = "Age",
       ylab = "Body Fat (%)")

abline(0, 1)

```

## OUTPUT:





**Q6.** Suppose that a hospital tested the age and body fat data for 18 randomly selected adults with the following results:

- Use min-max normalization to transform the value 35 for age onto the range [0.0, 1.0].
- Use z-score normalization to transform the value 35 for age, where the standard deviation of age is 12.94 years.
- Use normalization by decimal scaling to transform the value 35 for age. Perform the above functions using R – tool

**R CODE:**

```
v <- c(23,23,27,27,39,41,47,49,50,52,54,54,56,57,58,58,60,61)
```

```
x <- 35
```

```
min_max <- (x - min(v)) / (max(v) - min(v))
```

```
print(min_max)
```

```
m <- mean(v)
```

```
s <- 12.94
```

```
z_score <- (x - m) / s
```

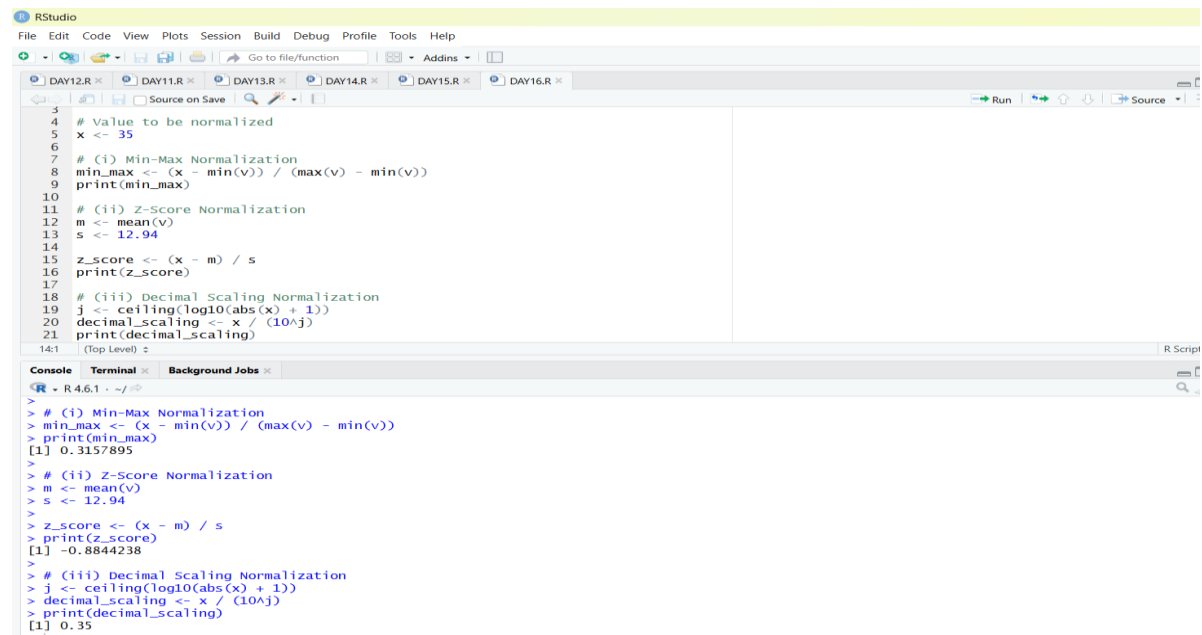
```
print(z_score)
```

```
j <- ceiling(log10(abs(x) + 1))
```

```
decimal_scaling <- x / (10^j)
```

```
print(decimal_scaling)
```

**OUTPUT:**



```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
DAY12.R DAY11.R DAY13.R DAY14.R DAY15.R DAY16.R
3
4 # Value to be normalized
5 x <- 35
6
7 # (i) Min-Max Normalization
8 min_max <- (x - min(v)) / (max(v) - min(v))
9 print(min_max)
10
11 # (ii) Z-Score Normalization
12 m <- mean(v)
13 s <- 12.94
14
15 z_score <- (x - m) / s
16 print(z_score)
17
18 # (iii) Decimal Scaling Normalization
19 j <- ceiling(log10(abs(x) + 1))
20 decimal_scaling <- x / (10^j)
21 print(decimal_scaling)
14:1 (Top Level) R Script

Console Terminal Background Jobs
> # (i) Min-Max Normalization
> min_max <- (x - min(v)) / (max(v) - min(v))
> print(min_max)
[1] 0.3157895
>
> # (ii) Z-Score Normalization
> m <- mean(v)
> s <- 12.94
>
> z_score <- (x - m) / s
> print(z_score)
[1] -0.8844238
>
> # (iii) Decimal Scaling Normalization
> j <- ceiling(log10(abs(x) + 1))
> decimal_scaling <- x / (10^j)
> print(decimal_scaling)
[1] 0.35
```

**Q7.** The following values represent the number of pencils available in different boxes. Create a vector and find the mean, median, and mode of the given data.

| Box     | Box1 | Box2 | Box3 | Box4 | Box5 | Box6 | Box7 | Box8 | Box9 | Box10 |
|---------|------|------|------|------|------|------|------|------|------|-------|
| Pencils | 9    | 25   | 23   | 12   | 11   | 6    | 7    | 8    | 9    | 10    |

**R CODE:**

```
pencils <- c(9,25,23,12,11,6,7,8,9,10)
```

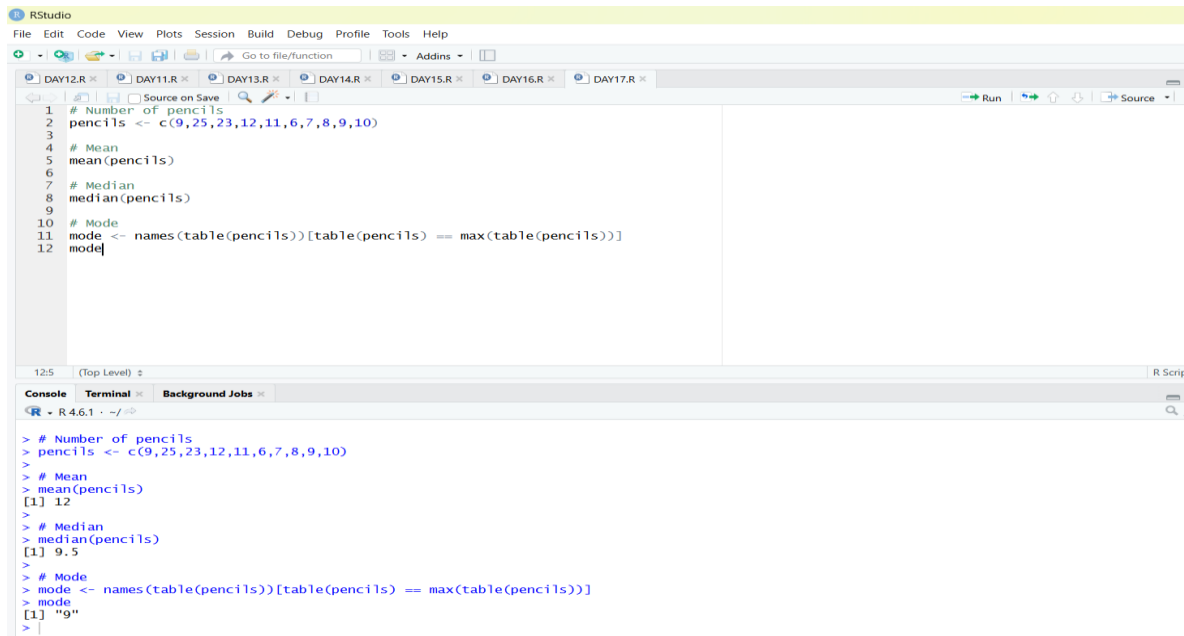
```
mean(pencils)
```

```
median(pencils)
```

```
mode <- names(table(pencils))[table(pencils) == max(table(pencils))]
```

```
mode
```

## OUTPUT:



The screenshot shows the RStudio interface. The source editor contains the following R code:

```
1 # Number of pencils
2 pencils <- c(9,25,23,12,11,6,7,8,9,10)
3
4 # Mean
5 mean(pencils)
6
7 # Median
8 median(pencils)
9
10 # Mode
11 mode <- names(table(pencils))[table(pencils) == max(table(pencils))]
12 mode
```

The console shows the output of the code:

```
> # Number of pencils
> pencils <- c(9,25,23,12,11,6,7,8,9,10)
>
> # Mean
> mean(pencils)
[1] 12
>
> # Median
> median(pencils)
[1] 9.5
>
> # Mode
> mode <- names(table(pencils))[table(pencils) == max(table(pencils))]
> mode
[1] "9"
```

**Q8.** The following table represents pairs of values (x, y), where the first column represents the number of mobile phones sold and the second column represents the money earned. Plot a scatter plot to study the relationship between the number of mobile phones sold and money.

|                        |    |   |    |    |    |   |     |    |     |     |
|------------------------|----|---|----|----|----|---|-----|----|-----|-----|
| x (Mobile Phones Sold) | 4  | 1 | 5  | 7  | 10 | 2 | 50  | 25 | 90  | 36  |
| y (Money)              | 12 | 5 | 13 | 19 | 31 | 7 | 153 | 72 | 275 | 110 |

## R CODE:

```
x <- c(4,1,5,7,10,2,50,25,90,36)
```

```
y <- c(12,5,13,19,31,7,153,72,275,110)
```

```
scatter.smooth(x, y,
```

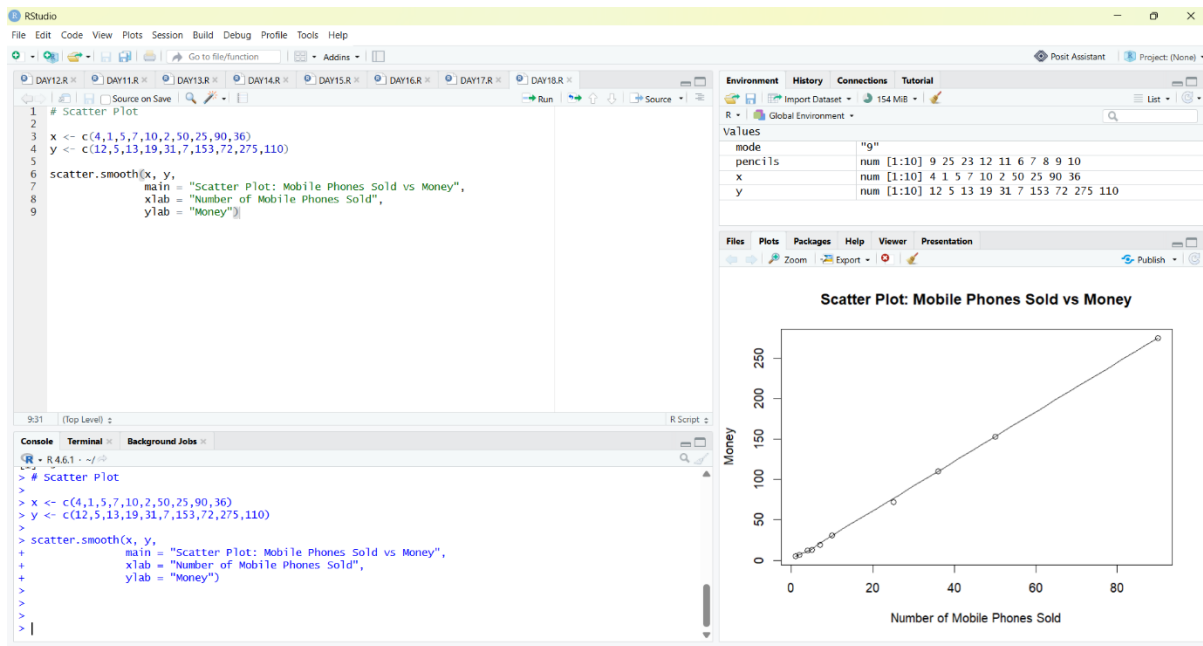
```
  main = "Scatter Plot: Mobile Phones Sold vs Money",
```

```
  xlab = "Number of Mobile Phones Sold",
```

```
  ylab = "Money")
```

## OUTPUT:





**Q9.** Implement of the R script using marks scored by a student in his model exam has been sorted as follows: 55, 60, 71,

63, 55, 65, 50, 55, 58, 59, 61, 63, 65, 67, 71, 72, 75. Partition them into three bins by each of the following methods. Plot the data

points using histogram.

(a) equal-frequency (equi-depth) partitioning (b) equal-width partitioning

**R CODE:**

```

marks <- c(55, 60, 71, 63, 55, 65, 50, 55, 58,
           59, 61, 63, 65, 67, 71, 72, 75)

num_bins <- 3

sorted_marks <- sort(marks)

bins_eq_frequency <- cut(
  sorted_marks,
  breaks = quantile(sorted_marks,
                    probs = seq(0, 1, length.out = num_bins + 1)),
  include.lowest = TRUE
)

print("Equal-Frequency Bins:")

```

```

print(bins_eq_frequency)

hist(sorted_marks,
      breaks = quantile(sorted_marks,
                        probs = seq(0, 1, length.out = num_bins + 1)),
      main = "Equal-Frequency (Equi-Depth) Partitioning",
      xlab = "Marks",
      ylab = "Frequency")

bins_eq_width <- cut(
  marks,
  breaks = num_bins,
  labels = FALSE,
  include.lowest = TRUE
)

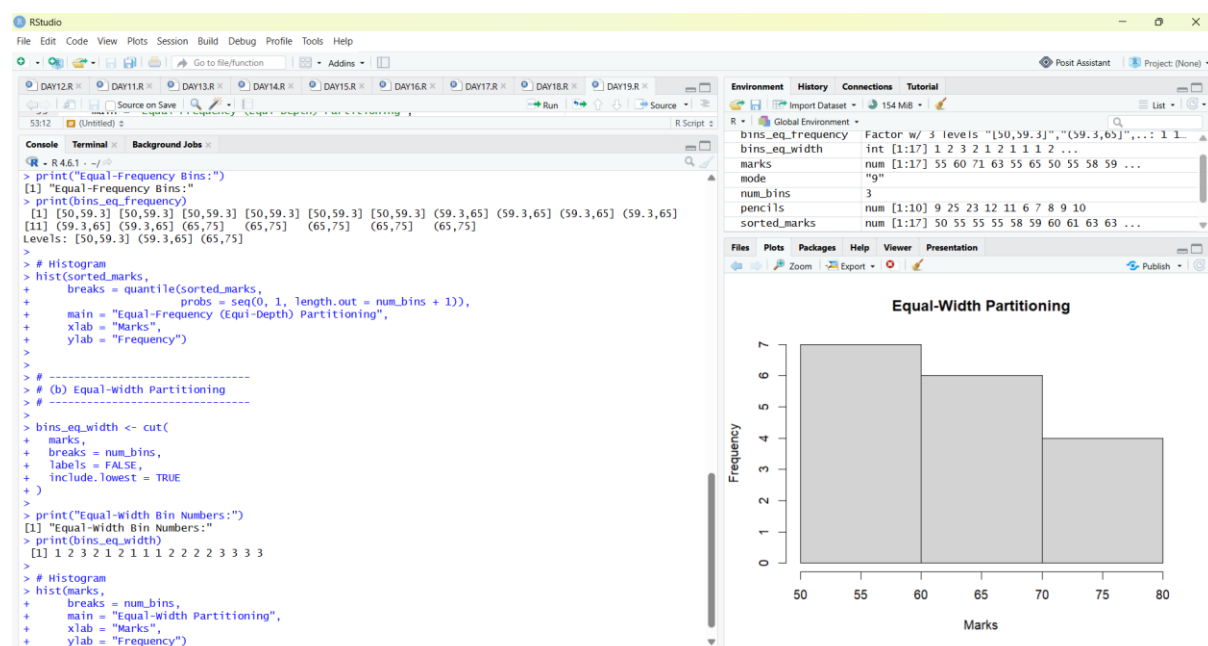
print("Equal-Width Bin Numbers:")

print(bins_eq_width)

hist(marks,
      breaks = num_bins,
      main = "Equal-Width Partitioning",
      xlab = "Marks",
      ylab = "Frequency")

```

## OUTPUT:



**Q10.** Suppose that the speed car is mentioned in different driving style.

**Regular 78.3 81.8 82 74.2 83.4 84.5 82.9 77.5 80.9 70.6 Speed**

**Calculate the Inter quantile and standard deviation of the given data.**

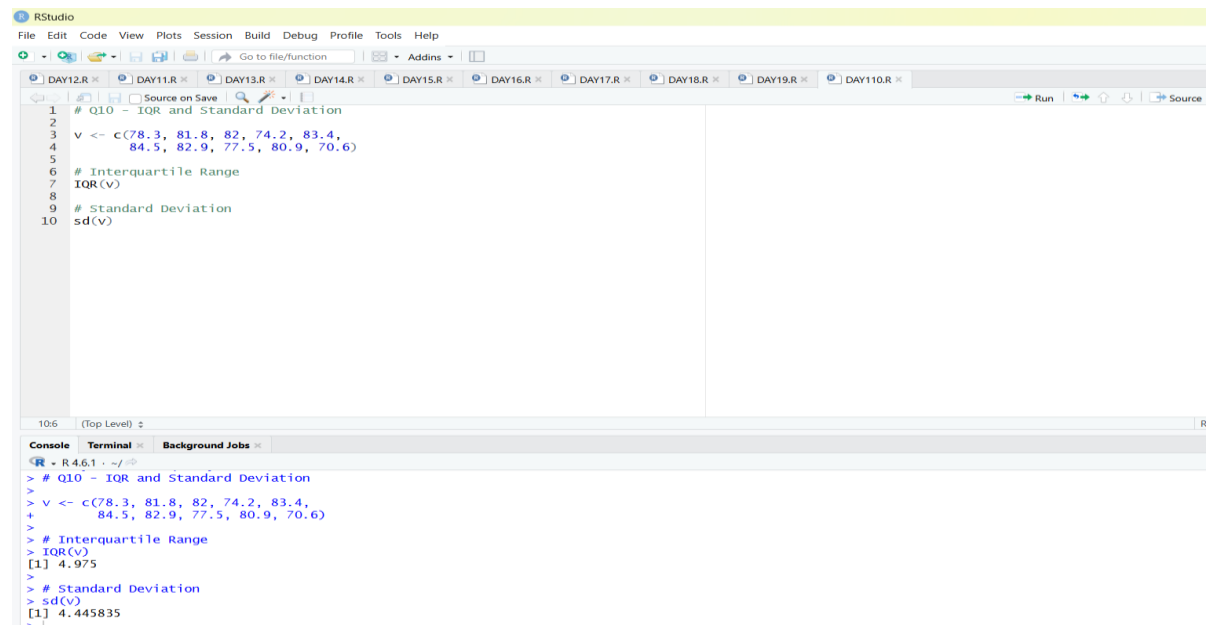
**R CODE:**

```
v <- c(78.3, 81.8, 82, 74.2, 83.4,  
      84.5, 82.9, 77.5, 80.9, 70.6)
```

IQR(v)

sd(v)

**OUTPUT:**



The screenshot shows the RStudio interface. The script editor contains the following code:

```
1 # Q10 - IQR and Standard Deviation  
2  
3 v <- c(78.3, 81.8, 82, 74.2, 83.4,  
4       84.5, 82.9, 77.5, 80.9, 70.6)  
5  
6 # Interquartile Range  
7 IQR(v)  
8  
9 # Standard Deviation  
10 sd(v)
```

The console output shows the execution of the code:

```
R - R 4.6.1 - ~/...  
> # Q10 - IQR and Standard Deviation  
>  
> v <- c(78.3, 81.8, 82, 74.2, 83.4,  
+       84.5, 82.9, 77.5, 80.9, 70.6)  
>  
> # Interquartile Range  
> IQR(v)  
[1] 4.975  
>  
> # Standard Deviation  
> sd(v)  
[1] 4.445835  
>
```

**Q11.** Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order) 13,

**15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70.**

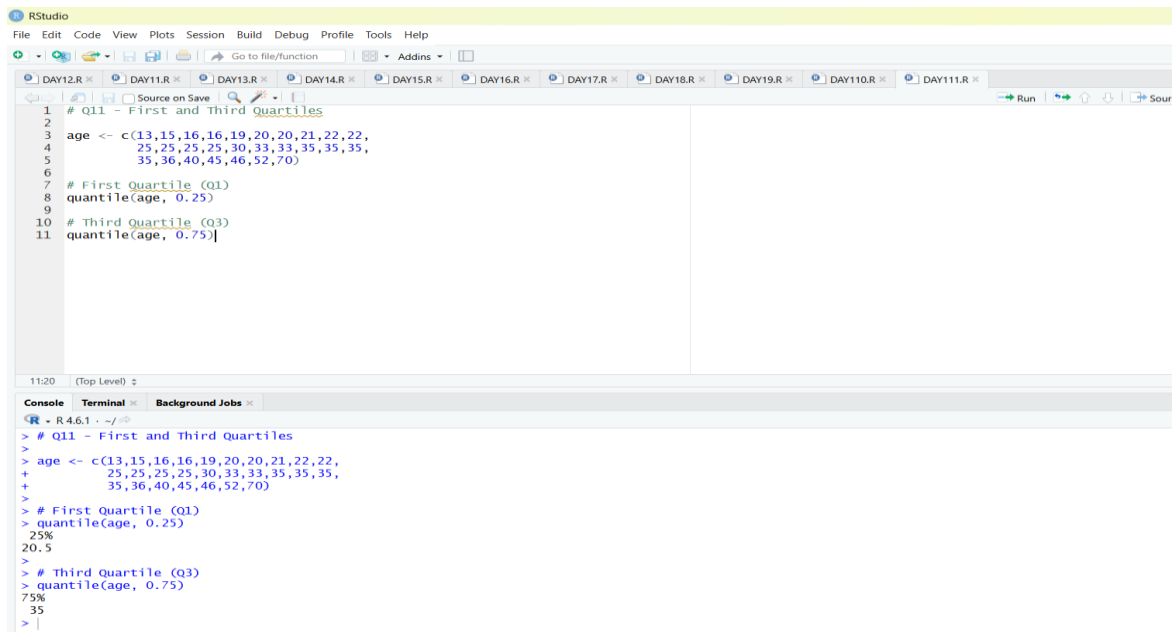
**Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data**

**R CODE:**

```
age <- c(13,15,16,16,19,20,20,21,22,22,  
        25,25,25,25,30,33,33,35,35,35,  
        35,36,40,45,46,52,70)  
  
quantile(age, 0.25)
```

quantile(age, 0.75)

## OUTPUT:



The screenshot shows the RStudio interface. The top menu bar includes File, Edit, Code, View, Plots, Session, Build, Debug, Profile, Tools, and Help. Below the menu is a toolbar with icons for running and saving. The main editor window displays the following R code:

```
1 # Q11 - First and Third Quartiles
2
3 age <- c(13,15,16,16,19,20,20,21,22,22,
4         25,25,25,25,30,33,33,35,35,35,
5         35,36,40,45,46,52,70)
6
7 # First Quartile (Q1)
8 quantile(age, 0.25)
9
10 # Third Quartile (Q3)
11 quantile(age, 0.75)
```

The bottom console window shows the output of the code:

```
> # Q11 - First and Third Quartiles
>
> age <- c(13,15,16,16,19,20,20,21,22,22,
+         25,25,25,25,30,33,33,35,35,35,
+         35,36,40,45,46,52,70)
>
> # First Quartile (Q1)
> quantile(age, 0.25)
25%
20.5
>
> # Third Quartile (Q3)
> quantile(age, 0.75)
75%
35
> |
```